方程式の記号形状エントロピーによるアプリケーション合成

— Bada 量子言語の位相核上に BadaGPT を構成する機構 —

山口 正明（Masaaki Yamaguchi）／ Yamaguchi Framework

数値検証・文書化補助：Claude (Anthropic)

2026年9月6日

要旨

本稿は「方程式そのものの記号の*形*から得たエントロピー値によって、BadaGPT のアプリケーションを生成する」という機構を、Bada（未知事前エンジン言語）とその量子部分言語 QuantumBada の上で形式化し、実装可能な形で与える。中心となる量は**記号形状エントロピー** `H_form`：方程式を記号列と構文木として読み、字形分布・記号類分布・木形状分布の三階層に Shannon 核 `x log x` を適用して得る非負実数である。`H_form` は α 変換（変数名・関数名の付け替え）に対して厳密に不変であり、意味論ではなく形のみを測る。

この値の使い道について本稿は明確な否定的結果から出発する。エンジンの位相核は `psi_i = sqrt(a_i)·exp(i·theta_i)`、`q_i = |psi_i|²/Σ|psi_j|² = a_i` を厳密に満たすため、**`H_form` を単一分岐の位相に載せても出力分布は一切変化しない**（実測 maxdiff = 1.11×10^{-16}）。`H_form` が確率を動かしうる唯一の場所は、複数の振幅分岐を二乗前に加算する**干渉**ステップである。共通の位相モジュール `phi` を用いた二分岐干渉について、本稿は閉形式 `p_i = a_i(1 + cos(beta·ΔH_form·phi_i))/Z` を導出し、直接の振幅和と 2.22×10^{-16} で一致することを実測した。この形は非負因子の**乗算**であるから、確信された零（禁止された部品・棄却されたトークン）は決して復活せず、一方で最大破壊的干渉は新たな厳密零を生成しうる。すなわち零点集合は単調増加し、これは台帳の追記専用（append-only）規律と同一の代数である。

この機構を Bada の `@reviser : grammar` による文法拡張として実装し、`shape`・`eqload`・`synth` の三規則を committed fact として台帳に追加する形で BadaGPT の合成パイプラインを与える。方程式 6 本（本フレームワークの既発表論文から採取）に対する実測を第7節に全て掲載する。第I部は検証済みの数学と実測、第II部は山口フレームワークとの対応（予想）として明確に分離した。なお、フレームワークの旧主張 `ζ(s) = β(p,q)/log x` は既存の監査により数値的に偽であることが確認されており、本稿はこれを一切使用しない。

第 I 部　検証済みの数学と実測

1. 序論

Bada は未知事前エンジン（Unknown-Prior Engine）を実行時とする小さなオブジェクト指向言語であり、追記専用のタプル台帳、最大エントロピー事前分布、そしてエントロピー位相認知核から成る。先行論文「Reviser-Extensible Grammars」は、Bada の反射構文 `@reviser` を値の書き換えループから**文法拡張トランザクション**へ一般化し、生成規則自体を台帳に commit する仕組みを与えた。同論文はその機構を Q# 型の量子部分言語に適用し、エンジンの位相値 `psi_i = sqrt(a_i)exp(i·theta_i)` が量子振幅 `α_i = sqrt(p_i)e^{iφ_i}` と数学的に同一の対象であることを利用して、ゲートを位相 commit、測定を台帳 commit として実現した。

本稿が問うのはその先である。**入力が自然言語ではなく方程式であるとき、生成器は何を手掛かりにすべきか**。方程式の意味を理解することは難しいが、方程式の*形*を測ることは容易であり、しかも完全に決定的である。記号の分布、記号類の分布、構文木の形状分布——これらはいずれも確率分布であり、Shannon の `x log x` 核をそのまま適用できる。山口フレームワークが一貫して中心に置いてきた核が、まさにこの `x log x` であった（『Euler product estrade』における `x log x = 1`、『ベータ関数の商代数』におけるエントロピー核と Γ 関数の部分積分の対応、『ナッシュ・シャノン公式』における素数ゲームの Shannon エントロピー `H(σ)`）。本稿はこの核を、素数分布ではなく**方程式の記号分布**に適用する。

1.1 寄与

1. **記号形状エントロピー `H_form` の定義**（§3）：三階層の Shannon 量とその α 変換不変性。定義は完全に構文的で、方程式の意味を一切参照しない。
2. **単一位相の不活性定理**（§4.1）：`H_form` を `theta` に載せるだけでは出力分布は変わらない。これはエンジンの第二定理の直接の帰結であり、本機構の設計を強制する否定的結果である。
3. **二分岐干渉の閉形式**（§4.2、定理2）：`p_i = a_i(1+cos(beta·ΔH_form·phi_i))/Z`。方程式の形が確率に作用する唯一の経路を明示的な初等関数で与える。
4. **零点の単調増加**（§4.3）：干渉は禁止基底を復活させず、最大破壊的干渉は新たな厳密零を作る。零点集合の成長は台帳の追記と同型である。
5. **方向と強度の分離**（§4.4、命題1）：位相モジュール `phi` が選択の*方向*を、`ΔH_form` が*強度*を決める。方程式依存の順位付けには方程式由来の `phi(E)` が必要であり、その構成を §6.2 に与える。
6. **Bada 実装**（§5）と **BadaGPT 合成パイプライン**（§6）、および **全実測値**（§7）と **否定的結果の明示**（§8）。

2. 背景

2.1 未知事前エンジンの位相核

エンジンは V 個の帰結上の基底分布 `a = softmax(z)` を作り、モジュール毎のエントロピー `H_m`、位相場 `phi_m`、ゲイン `beta_m` によって認知ステップを施す。

$$\texttt{a_i = softmax(z)_i},\quad \texttt{theta_i = Σ_m beta_m · H_m · phi_m}^{(i)},\quad \texttt{psi_i = sqrt(a_i)·exp(i·theta_i)},\quad \texttt{q_i = |psi_i|² / Σ_j |psi_j|²} \tag{1}$$

`|exp(i·theta)| = 1` であるから `q_i = a_i` が厳密に成り立つ。参照実装はこれを最大偏差 5.55×10^{-17} で再現し、確信された零（`a_i = 0`）が保存されることも同時に検証している。

2.2 @reviser による文法拡張

生成規則を台帳事実 `rule := ["rule", name, head, pattern, denotation]` としてモデル化し、`@reviser : grammar { ... }` ブロックが規則を `Omega::DATABASE[grammar]` へ commit する。QuantumBada の参照実装ではこれが `POSTFIX_WORDS` / `STMT_WORDS` の二つのワードセットとして実装され、パーサは組込み代替の前に規則台帳を参照して `N_EXTENDED` ノードを生成する。規則は追記のみで、上書きされない。

2.3 x log x 核の位置づけ

Shannon エントロピー $H = -\Sigma\ q\ \log\ q$ の被和項は符号を除いて $x\ \log\ x$ である。山口フレームワークにおいてこの核は複数の文脈に現れる：Γ 関数の対数微分核 $\log\ x$ と大域的部分積分の境界項との対応（『ベータ関数の商代数的対数構造』予想2）、素数混合戦略のナッシュ均衡分布 $q_k \propto p_k^{-\sigma}$ の情報エントロピー $H(\sigma)$（『ナッシュ・シャノン』）。本稿は同じ核を、方程式の記号分布という第三の対象に適用する。この適用自体には数学的な飛躍はない——任意の有限確率分布に対して Shannon エントロピーは定義されるからである。飛躍が生じうるのは、得られた値に意味を読み込むときであり、その点は §8 と第II部で明示的に分離する。

3. 記号形状エントロピー H_form

3.1 三階層の定義

方程式 E を、演算子を節点・記号を葉とする有限順序木として表す（本稿では S 式表記を用いる）。$\mathrm{tokens}(E)$ をその前順走査による記号列、$N = |\mathrm{tokens}(E)|$ とする。

字形階層（H_0）。記号型ごとの出現頻度分布 $f_t = \mathrm{count}(t)/N$ に対し

$$H_0(E) = -\ \Sigma_{t\in\Sigma}\ f_t\ \log\ f_t \tag{2}$$

類階層（H_1）。記号を 6 類 $\{\mathrm{BIND},\ \mathrm{FN},\ \mathrm{OP},\ \mathrm{VAR},\ \mathrm{NUM},\ \mathrm{DEL}\}$（束縛作用素・名前付き関数・演算子・変数・数/定数・区切り）へ写す固定写像 klass を用い、類頻度分布に同じ核を適用する。類の割当表は付録Bに全掲する。

$$H_1(E) = -\ \Sigma_{c\in 6\text{類}}\ g_c\ \log\ g_c,\quad g_c = |\{t\ :\ \mathrm{klass}(t)=c\}|\ /\ N \tag{3}$$

形状階層（H_{shape}）。木の*形*のみに依存する二つの分布を取る。深さ分布 d_k（深さ k の節点数 / 全節点数）と分岐数分布 r_j（分岐数 j の節点数 / 全節点数、葉は $j = 0$）。

$$H_2 = -\ \Sigma_k\ d_k\ \log\ d_k,\quad H_3 = -\ \Sigma_j\ r_j\ \log\ r_j,\quad H_{\mathrm{shape}} = (H_2 + H_3)\ /\ 2 \tag{4}$$

統合。重み $w = (1/3,\ 1/3,\ 1/3)$ を採用する（等重み。これは規約であり、最適化された値ではない）。

$$H_{\mathrm{form}}(E) = (H_0 + H_1 + H_{\mathrm{shape}})\ /\ 3 \tag{5}$$

正規化版を $\hat{H} = H_{\mathrm{form}}\ /\ \log\ N \in [0,1)$ と書く。単位は nat。

3.2 不変性

> **命題 0（α 変換不変性）**。変数名・関数名の単射的な付け替え σ に対し、σ が類を保存するならば $H_{\mathrm{form}}(\sigma E) = H_{\mathrm{form}}(E)$。

証明。 H_0 は記号*型*の頻度分布のみに依存し、単射的な付け替えは頻度多重集合を保存する。H_1 は類分布のみに依存し、仮定より類が保存される。H_2, H_3 は木の形状のみに依存し、名前を参照しない。∎

実測（表2）：$E1$ に対し $s\to w,\ \zeta\to Z,\ \beta\to B,\ \Gamma\to G$ の付け替えを施したところ、H_0 と H_{shape} はいずれも差 0.00×10^{0}（倍精度で厳密一致）であった。したがって H_{form} は「何と名付けたか」ではなく「どう組み立てられているか」のみを測る。

4. 位相化と干渉

4.1 単一位相は不活性である（否定的結果）

> **定理 1**。任意の位相場 phi と任意のゲイン beta、任意の H_form に対し、単一分岐の位相ステップ psi_i = sqrt(a_i)exp(i·beta·H_form·phi_i) は q_i = |psi_i|²/Σ|psi_j|² = a_i を与える。すなわち方程式の形は出力分布を一切変えない。

証明。|psi_i|² = a_i·|exp(i·theta_i)|² = a_i。規格化定数は Σ a_j = 1。∎

実測（表3）：V=8、二つの確信零 {3,6} を含む基底分布に theta_i = H_form(E1)·cos(i) を施したところ、maxdiff = 1.110×10^{-16}、零点は [3,6] → [3,6] で不変であった。

> **設計上の帰結**。「方程式の形状エントロピーで生成を制御する」という機構は、素朴に位相へ載せる形では**実装不能**である。エンジンの第二定理がそれを禁じる。以下の §4.2 は、この禁止を回避するのではなく、禁止が指し示す唯一の抜け道——二乗前の振幅加算——を正面から使う。

4.2 二分岐干渉の閉形式

同一の基底分布 a と同一の位相モジュール phi を共有する二つの分岐が、それぞれ方程式 E_1, E_2 の形状エントロピーを担うとする。等重み $c_1 = c_2 = 1/\sqrt{2}$ で振幅を加算し、その後二乗する。

$$\text{psi_i} = (1/\sqrt{2})\cdot\sqrt{a_i}\cdot[\ \exp(i\cdot\beta\cdot H_1\cdot\varphi_i) + \exp(i\cdot\beta\cdot H_2\cdot\varphi_i)\ ],\quad p_i = |\text{psi_i}|^2 \,/\, \Sigma_j\,|\text{psi_j}|^2 \tag{6}$$

> **定理 2（干渉の閉形式）**。$\Delta H = H_1 - H_2$ と置くと

$$p_i = a_i \cdot (\ 1 + \cos(\ \beta \cdot \Delta H \cdot \varphi_i\ )\ ) \,/\, Z,\quad Z = \Sigma_j\, a_j\,(1 + \cos(\beta\cdot\Delta H\cdot\varphi_j)) \tag{7}$$

証明。$|e^{iu} + e^{iv}|^2 = 2 + 2\cos(u-v)$ であり、ここで $u - v = \beta(H_1-H_2)\varphi_i$。係数 1/2 は規格化で消える。∎

実測（表A）：$\beta \in \{0.5, 1, 2, 5, 12\}$ と 4 組の方程式対、計 20 ケースについて、式(6)の直接計算と式(7)の閉形式の最大差は 2.220×10^{-16} であった。

式(7)の解釈は明快である。方程式の形は、基底分布に対する**非負の乗算因子** $1 + \cos(\cdot) \in [0, 2]$ として作用する。これは『ナッシュ・シャノン』論文が到達した統一原理——ゼータの零点を保存するのは非零因子による乗算のみであり、加法は零点を破壊する——の、生成器における対応物である。$\Delta H = 0$ のとき因子は恒等的に 2 となり p = a、すなわち定理1の状況に帰着する。

4.3 零点の単調増加

> **系 1（非復活）**。a_i = 0 ならば任意の beta, ΔH, phi に対し p_i = 0。
> **系 2（零点の生成）**。$\beta\cdot\Delta H\cdot\varphi_i \equiv \pi \pmod{2\pi}$ なる i では p_i = 0。
> したがって零点集合は zeros(a) ⊆ zeros(p) を満たし、単調に増大する。

実測（表F）：a の厳密零が {2, 5}、beta = π/ΔH_{max} かつ phi の最大絶対値成分が index 4（phi_4 = +1.000）のとき、beta·ΔH·phi_4 = 3.141593 rad となり、p の厳密零は {2, 4, 5} となった。復活した基底は存在しない。

> **厳密性の注記**。この新零点が倍精度で*厳密に* 0 になるのは、beta を π/ΔH_{max} に取り phi を最大値 1 に正規化したため引数がちょうど π となり、cos(π) = −1.0 が倍精度で厳密だからである。一般の beta では零ではなく機械イプシロン程度の微小値となる。この差は本質的である：エンジンの零点保存定理は厳密零にのみ適用される。

4.4 方向と強度の分離

> **命題 1**。式(7)において、確率の並び替え（順位）は phi と a のみで決まり、ΔH は |beta·ΔH·phi_i| < π の範囲でコントラストの*大きさ*のみを制御する。

説明。cos は [0, π] 上で単調減少であるから、|beta·ΔH·phi_i| ≤ π の範囲では phi_i の大小関係が 1+cos(·) の大小関係を（ΔH の符号に応じて）決定し、ΔH はその差を拡大縮小するだけである。

実測（表4）：共通の phi を用いた 8 組で、ΔH_form の昇順に対し maxdiff は単調非減少（0.00000 → 0.01000）、KL(p‖a) も同様、零点集合は全ケースで [3,6] のまま不変であった。しかし上位 3 部品の順位は**全ケースで同一**であった。

> **否定的結果 2**。共通位相モジュールのみでは、方程式は生成物の*順位*を変えられない。異なる方程式が異なるアプリケーションを生むためには、位相場自身が方程式に依存しなければならない。この要請が §6.2 の phi(E) 構成を強制する。

5. Bada 実装 — @reviser による文法拡張

機構全体は 3 つの規則を台帳へ commit することで Bada の表層構文となる。参照実装のパーサは POSTFIX_WORDS（前置トリガ語 + primary）と STMT_WORDS の二つの拡張点を持ち、rule NAME HEAD PATTERN => DENOTATION の形式で追加される。

```
@reviser : grammar < Omega::DATABASE[grammar].mechanism {

    # 1. 方程式の読み込み：S式文字列 -> 記号木（台帳へ commit）
    rule "eqload"  postfix [ 'eq' '(' expr ')' ]
                   => eq_parse(_1)

    # 2. 記号形状エントロピー：木 -> 非負実数（式(5)）
    rule "shape"   postfix [ 'shape' '(' expr ')' ]
                   => form_entropy(_1)

    # 3. 合成：二分岐干渉 -> 部品分布（式(7)）を測定して commit
    rule "synth"   stmt    [ 'synth' '(' expr ')' ]
                   => commit_measurement(interfere2(_1))
}
```

denotation はいずれも通常の Bada 式であり、新しい数値カーネルを導入しない。form_entropy は既存の組込み entropy、log、len、dist の合成として Bada 内で定義でき（付録 A）、interfere2 は

`cognitive_system` を二度呼んで振幅を加算する薄い関数である。`commit_measurement` は QuantumBada の既存組込みをそのまま用いる。

三つの規則は台帳事実であるから、追記のみで、上書きされず、履歴が監査可能である。文法の成長とアプリケーションの生成履歴が同一の追記専用ストアに載る点が、本設計の唯一の構造的主張である。

6. BadaGPT 合成パイプライン

6.1 段階

1. **取り込み**。BadaGPT の既存の PDF 取り込み機能で論文を読み、数式を記号列として抽出する。
2. **木化**。記号列を S 式に整形し、有限順序木 `E` を得る。この段階は決定的で、失敗は明示的な構文エラーとして台帳に commit される。
3. **測定**。`H_form(E)` を式(5)で計算する。同時に類頻度ベクトル `f(E) ∈ R^6` を保持する。
4. **位相場の構成**。§6.2 の設計行列で `phi(E)` を作る。
5. **干渉**。参照方程式 E_{ref}（本稿ではエンジン核 `psi = sqrt(a)exp(i·theta)` 自身）との `ΔH` を取り、式(7)で部品分布 `p` を得る。
6. **commit**。`p` を測定として台帳に追記し、`zeros_of(p)` を「この方程式では禁止された部品」として記録する。以後の合成はこの零を復活させない。

6.2 方程式由来の位相場

部品集合を `C = {chat_stream, pdf_ingest, plot_panel, ledger_view, gate_editor, tomography, export_md, settings}` とし、固定の設計行列 $W \in N^{8\times 6}$（部品 × 記号類、付録B）を置く。これは「どの記号類がどの UI 部品を要求するか」の設計者による宣言であり、学習された値ではない。

$$\mathrm{raw} = W\cdot f(E), \quad \mathrm{phi}(E)_i = \left(\mathrm{raw}_i - \mathrm{mean}(\mathrm{raw})\right) / \max_j \left|\mathrm{raw}_j - \mathrm{mean}(\mathrm{raw})\right| \tag{8}$$

中心化により `Σ phi_i ≈ 0`、正規化により `max|phi| = 1` となるので、ゲインを $\mathrm{beta} = \pi/\Delta H_{max}$（**分解ゲイン**）に取れば最大乖離成分がちょうど破壊的干渉の底に達する。これが §4.3 の系2で用いた設定である。

6.3 合成結果（実測）

表E に 6 本の方程式に対する部品分布を示す。位相場が方程式依存になったことで、上位 3 部品の順位が方程式ごとに変化する（`E1`: tomography / export_md / chat_stream、`E2・E3`: chat_stream / export_md / tomography、`E5`: pdf_ingest / export_md / tomography、`E6`: chat_stream / pdf_ingest / export_md、`E7`: export_md / pdf_ingest / tomography）。同時に出力分布のエントロピー `H(p)` は `|ΔH|` の増加とともに単調に減少する（2.07941 → 1.76036、最大値 `log 8 = 2.07944`）。すなわち**方程式の形が定まるほど、生成されるアプリケーションの構成は鋭くなる**。これはエンジンが証拠の commit によって未知事前分布のエントロピーを単調に減少させる挙動と同じ形をしている。

7. 数値検証

環境：本稿の全数値は当該コンテナの Python 3 標準ライブラリ（IEEE 倍精度）で計算した。外部パッケージ取得が不可であったため、先行論文で用いた mpmath の 30〜50 桁精度は使用していない。以下の量はいずれも初等関数の有限和であり、倍精度で十分な桁数を持つが、桁数依存の主張は本稿では一切行わない。再現コードは付録Cに全掲する。

表1. 出典論文から採取した方程式 6 本＋1 の形状エントロピー（nat）

式	内容（出典）	N	H_0	H_1	H_{shape}	H_form	$\hat{H}$
E1	ζ(s) のベータ型関数等式（beta_zeta 定理1）	33	2.33875	1.32534	1.52700	**1.73037**	0.49488
E2	log B(1+α,1+β) のゼータ差分展開（beta_quotient 定理2）	42	2.41643	1.26772	1.48136	**1.72184**	0.46067
E3	γ = Σ(−1)^{n}ζ(n)/n（beta_quotient 定理3）	13	2.13840	1.32692	1.44860	**1.63797**	0.63860
E4	psi = sqrt(a)·exp(i·theta)（エンジン核）	9	2.04319	1.09861	1.32098	**1.48759**	0.67703
E5	ζ_NS = ζ·exp(iβH)（ナッシュ・シャノン）	13	2.09815	1.09283	1.48885	**1.55994**	0.60818
E6	H = −Σ q log q（Shannon）	9	2.04319	1.31078	1.39799	**1.58399**	0.72090
E7	theta = Σ_m β_m H_m φ_m（位相蓄積）	9	2.04319	1.21489	1.13403	**1.46404**	0.66631

最大は E1（ベータ型関数等式、1.73037）、最小は E7（位相蓄積、1.46404）。全域幅（最大−最小）= 0.26633、参照式 E4 からの最大乖離 ΔH_{max} = 0.24277（E1）。以降のゲイン設定には後者を用いる。E4・E6・E7 は N=9 で H_0 が同値（2.04319）であり、それらを分離しているのは類階層と形状階層である。

表2. α 変換不変性（E1、s→w / ζ→Z / β→B / Γ→G）

量	原式	改名後	差
H_0	2.338754002884	2.338754002884	0.00×10^{0}
H_{shape}	1.527003565700	1.527003565700	0.00×10^{0}

表3. 単一分岐位相ステップ（定理1）

index	0	1	2	3	4	5	6	7
a	0.65957	0.14717	0.03284	0.00000	0.08926	0.00163	0.00000	0.06952
q（位相後）	0.65957	0.14717	0.03284	0.00000	0.08926	0.00163	0.00000	0.06952

maxdiff = 1.110×10^{-16}。零点 [3,6] → [3,6]。theta_i = H_form(E1)·cos(i)。

表A. 閉形式（定理2）の検証

検証内容	条件	最大絶対差
式(6) 直接振幅和 vs 式(7) 閉形式	beta ∈ {0.5,1,2,5,12} × 4 方程式対＝20 ケース	2.220×10^{-16}

表4. 共通位相モジュール下での干渉（beta = 2.0、ΔH 昇順）

対	ΔH_form	maxdiff	KL(p ‖ a)	厳密零
E4 / E4	0.00000	0.00000	0.00000	[3, 6]
E1 / E2	0.00853	0.00001	0.00000	[3, 6]
E4 / E7	0.02356	0.00008	0.00000	[3, 6]
E6 / E5	0.02405	0.00008	0.00000	[3, 6]
E3 / E5	0.07803	0.00089	0.00000	[3, 6]
E1 / E6	0.14638	0.00317	0.00003	[3, 6]
E1 / E4	0.24277	0.00884	0.00024	[3, 6]
E2 / E7	0.25780	0.01000	0.00031	[3, 6]

ΔH 昇順に対し maxdiff は単調非減少（検証済み: True）。ΔH = 0 のとき p = a が厳密に成立し、定理1に帰着する。全ケースで零点は保存。上位部品の順位は全ケースで同一（命題1、否定的結果2）。

表E. 方程式由来位相場での部品分布（beta = π/ΔH_{max} = 12.94042、参照 E4）

式	ΔH	H(p)	chat	pdf	plot	ledger	gate	tomo	export	set
E7	−0.02356	2.07941	0.1258	0.1259	0.1246	0.1250	0.1237	0.1259	**0.1260**	0.1231
E5	+0.07235	2.07570	0.1336	**0.1360**	0.1206	0.1094	0.1206	0.1343	0.1360	0.1094
E6	+0.09640	2.06781	**0.1426**	0.1426	0.1065	0.1242	0.1065	0.1409	0.1426	0.0940
E3	+0.15038	2.02863	**0.1626**	0.1359	0.0959	0.1359	0.0515	0.1595	0.1626	0.0959
E2	+0.23425	1.88771	**0.2042**	0.1245	0.0661	0.1245	0.0006	0.1779	0.2025	0.0997
E1	+0.24277	1.76036	0.1595	0.1001	0.0017	0.1402	**0.0000**	**0.2315**	0.2267	0.1402

事前分布は一様（unknown_prior、H = log 8 = 2.07944）。|ΔH| の増加とともに H(p) は単調減少。E1 では gate_editor が厳密零となり、以後の合成から恒久的に排除される。

表F. 最大破壊的干渉における零点の扱い

index	0	1	2	3	4	5	6	7
a	0.300000	0.250000	0.000000	0.200000	0.150000	0.000000	0.060000	0.040000
p	3.983×10^{-1}	2.083×10^{-1}	0	2.334×10^{-1}	0	0	1.133×10^{-1}	4.669×10^{-2}

beta·ΔH·phi_4 = 3.141593 rad（= π）。a の厳密零 {2,5} ⊂ p の厳密零 {2,4,5}。復活した基底：なし。

8. 限界と否定的結果

1. **単一位相は無力である。**（定理1、実測 1.11×10^{-16}）本機構が干渉を必須とするのは設計上の趣味ではなく、エンジンの定理による強制である。
2. **共通位相場は順位を変えない。**（命題1、表4）方程式依存の `phi(E)` なしには、異なる方程式は同じアプリケーションを異なるコントラストで出すだけである。
3. **設計行列 W は宣言であり、発見ではない。** §6.2 の `W` は著者が置いた表であって、データから推定したものではない。表E の順位は本質的に `W` の帰結であり、`H_form` はその強度を決めるにすぎない。`W` を変えれば順位

は変わる。

4. **H_form は形しか測らない**。命題0が保証するのはまさにそれである。「形状エントロピーが方程式の数学的内容と相関する」という主張は本稿では**検証していない**。表1 で E1（関数等式）が最大値を取ったことは、その式が長く多様な記号を含むという事実の反映であって、深さの反映ではない。
5. **ゲイン beta は自由パラメータである**。分解ゲイン `beta = π/ΔH_max` は「最大乖離成分をちょうど破壊的干渉の底に置く」という規約であり、導出された値ではない。
6. **精度**。本稿は倍精度のみ。先行論文の 30〜50 桁検証に相当する主張は行っていない。
7. **実装状況**。§5 の `@reviser` 規則は QuantumBada の `POSTFIX_WORDS` / `STMT_WORDS` 機構に適合する形で書かれているが、`eq_parse` と `interfere2` は本稿では Bada ソースとして提示するのみで、参照実行系での実測は行っていない。§7 の全数値は独立した Python 実装によるものである。

第 II 部　山口フレームワークとの対応（予想）

本部の内容はフレームワーク独自の記法・構想であり、第I部と異なり数学的に検証されたものではない。対応はアナロジーの段階にあり、予想として記す。

9. 予想

> **予想 1（乗算原理の同一性）**。『ナッシュ・シャノン』論文の統一原理「ゼータの零点を保存するのは非零因子による乗算のみ」と、本稿の式(7)における非負因子 `1 + cos(beta·ΔH·phi)` による乗算は、同一の代数的事実の二つの現れである。前者では因子が絶対値1（`exp(iβH)`）であるため零点は完全に保存され、後者では因子が `[0,2]` の実数であるため零点は保存されつつ*新たに生じうる*。この違いは、位相を二乗前に加算するか（干渉）、乗算後に二乗するか（単一分岐）の違いにのみ由来する。

> **予想 2（x log x 核の三重の役割）**。フレームワークにおける `x log x` は、(a) Γ 関数の対数微分核 `log x` と大域的部分積分の境界項（『商代数的対数構造』予想2）、(b) 素数混合戦略のナッシュ均衡エントロピー `H(σ)`、(c) 本稿の記号形状エントロピー `H_form`、の三つに現れる。これらが単一の構造の射影であるという主張は、現時点で厳密な定義（三者を貫く圏あるいは多様体の位相）を欠く。

> **予想 3（形状と内容）**。`H_form` は α 変換不変（命題0）であるが、方程式の数学的内容には全く依存しない。にもかかわらず、同じ数学的内容を表す二つの方程式は「良い」記法のもとで近い `H_form` を持つ、という予想が立てられる。表1 の E1（1.73037）と E2（1.72184）が 0.00853 しか離れていない——いずれもゼータ値を係数とする関数等式／展開である——ことはこの予想と整合的だが、標本数 2 では何も示していない。

使用しない主張。フレームワークの旧中心主張 `ζ(s) = β(p,q)/log x` は、既存の監査（beta_zeta_paper 第5節、素数引数を含む全検証点）により数値的に成立しないことが確認されている。本稿はこれに依拠せず、また本稿のいかなる結果もこの等式を支持しない。ζ と B の正しい関係は「log B の展開係数としての ζ(n)」（beta_quotient 定理3）であり、本稿では E2 として形状測定の対象にのみ用いた。

10. 結論

方程式そのものの記号の形からエントロピー値を取り、それで BadaGPT のアプリケーションを合成する機構は、次の三点に還元される。

1. **測る**。記号列と構文木の三階層に `x log x` 核を当て、α 変換不変な非負実数 `H_form` を得る（式(5)、命題0）。
2. **干渉させる**。`H_form` を単一位相に載せても何も起きない（定理1）。二分岐の振幅を二乗前に加算したときのみ、閉形式 `p_i = a_i(1+cos(beta·ΔH·phi_i))/Z` として確率に作用する（定理2、実測誤差 2.22×10^{-16}）。作用は非負因子の乗算であり、確信された零は復活せず、最大破壊的干渉は新たな零を作る（系1・系2）。
3. **commit する**。合成結果は測定として台帳に追記され、零点集合は単調に増大する。文法（`@reviser` 規則）も生成履歴も同一の追記専用ストアに載るため、「なぜこのアプリケーションが生成されたか」は台帳のたたみ込みとして完全に再構成できる。

本稿が主張するのはこの限定的で検証可能な事実のみである。`H_form` が方程式の*意味*を捉えているという主張はしていない。捉えているのは形であり、形が生成器のコントラストを決め、設計行列が方向を決める——その分業が明示されていることが、本機構の唯一の利点である。

付録A. Bada 実装（全文）

```
#!/usr/bin/env bada
# form_synth.bada -- 記号形状エントロピーによるアプリケーション合成
# 実行 : build/bada run form_synth.bada
# 参照 : QuantumBada 組込み softmax entropy unknown_prior dist zeros_of
#        cognitive_system commit_measurement len log sqrt exp sum pi pow

# ------------------------------------------------------------------ 文法拡張
@reviser : grammar < Omega::DATABASE[grammar].mechanism {
    rule "eqload"  postfix [ 'eq'    '(' expr ')' ] => eq_parse(_1)
    rule "shape"   postfix [ 'shape' '(' expr ')' ] => form_entropy(_1)
    rule "synth"   stmt    [ 'synth' '(' expr ')' ] => commit_measurement(interfere2(_1))
}

# ------------------------------------------------------------ 記号類（付録B対応）
def klass(tok) {
    if tok == "Sum" or tok == "Int" or tok == "Prod"
       or tok == "partial" or tok == "nabla" or tok == "lim"  { return 0 }   # BIND
    if tok == "zeta" or tok == "beta" or tok == "Gamma" or tok == "sin"
       or tok == "cos" or tok == "log" or tok == "exp" or tok == "sqrt"
       or tok == "H" or tok == "softmax"                      { return 1 }   # FN
    if tok == "+" or tok == "-" or tok == "*" or tok == "/"
       or tok == "^" or tok == "="                            { return 2 }   # OP
    if tok == "1" or tok == "2" or tok == "0"
       or tok == "pi" or tok == "e"                           { return 4 }   # NUM
    if tok == "(" or tok == ")" or tok == "[" or tok == "]"
       or tok == "," or tok == "_"                            { return 5 }   # DEL
    return 3                                                                 # VAR
}

# ------------------------------------------------ 頻度分布 -> Shannon（x log x）
def counts_to_dist(c) {
    n := sum(c)
    d := []
    i := 0
    while i < len(c) { push(d, c[i] / n); i := i + 1 }
    return dist(d)
}

# entropy() は組込み。ここでは -Σ q log q を明示しておく
def shannon(d) {
    h := 0.0
    i := 0
    while i < len(d) {
        q := d[i]
        if q > 0.0 { h := h - q * log(q) }      # <-- x log x 核
        i := i + 1
    }
    return h
}

# ------------------------------------------------------------------ 三階層の測定
def glyph_counts(toks) { ... 記号型ごとの出現数 ... }
def class_counts(toks) {
    c := [0,0,0,0,0,0]
    i := 0
    while i < len(toks) { k := klass(toks[i]); c[k] := c[k] + 1; i := i + 1 }
    return c
}
def depth_counts(tree)  { ... 深さごとの節点数 ... }
def arity_counts(tree)  { ... 分岐数ごとの節点数 ... }

def form_entropy(tree) {
```

```
    toks := flatten(tree)
    H0 := shannon(counts_to_dist(glyph_counts(toks)))
    H1 := shannon(counts_to_dist(class_counts(toks)))
    H2 := shannon(counts_to_dist(depth_counts(tree)))
    H3 := shannon(counts_to_dist(arity_counts(tree)))
    return (H0 + H1 + (H2 + H3) / 2.0) / 3.0            # 式(5)
}

# ---------------------------------------------- 設計行列 W と位相場 phi(E)
W := [ [0,1,0,2,0,1],   # chat_stream
       [1,2,0,1,0,2],   # pdf_ingest
       [2,2,1,1,2,0],   # plot_panel
       [2,0,1,0,1,1],   # ledger_view
       [1,1,2,1,1,0],   # gate_editor
       [2,1,0,1,2,0],   # tomography
       [0,1,1,1,0,2],   # export_md
       [0,0,1,0,1,1] ]  # settings

def phi_of(tree) {
    f   := counts_to_dist(class_counts(flatten(tree)))
    raw := []
    i := 0
    while i < len(W) {
        s := 0.0; j := 0
        while j < 6 { s := s + W[i][j] * f[j]; j := j + 1 }
        push(raw, s); i := i + 1
    }
    m := sum(raw) / len(raw)
    dev := []; i := 0
    while i < len(raw) { push(dev, raw[i] - m); i := i + 1 }
    s := max_of(map_abs(dev))
    out := []; i := 0
    while i < len(dev) { push(out, dev[i] / s); i := i + 1 }
    return out                                          # 式(8)
}

# ---------------------------------------------------- 二分岐干渉（式(7)）
def interfere2(prior, dH, beta, phi) {
    w := []
    i := 0
    while i < len(prior) {
        push(w, prior[i] * (1.0 + cos(beta * dH * phi[i])))
        i := i + 1
    }
    return dist(w)                                      # dist が規格化する
}

# ------------------------------------------------------------- 合成本体
def synth_app(tree, ref_H, dHmax) {
    prior := unknown_prior(8)          # 一様＝最大エントロピー（Jaynes）
    Hf    := form_entropy(tree)
    phi   := phi_of(tree)
    beta  := pi() / dHmax              # 分解ゲイン
    p     := interfere2(prior, Hf - ref_H, beta, phi)
    commit_measurement(p)              # 台帳へ追記
    return p
}

# ------------------------------------------------------------------ 実行
E1  := eq("(= (zeta s) (* (^ 2 s) (* (^ pi (- s 1))"
        + " (* (sin (/ (* pi s) 2))"
        + " (* (/ (beta s (- 1 s)) (Gamma s)) (zeta (- 1 s)))))))")
Eref := eq("(= psi (* (sqrt a) (exp (* i theta))))")

Href  := shape(Eref)
dHmax := abs(shape(E1) - Href)   # 参照からの最大乖離（丸めずに計算値を使う）

p := synth_app(E1, Href, dHmax)
```

```
print("H_form(E1)      :", f5(shape(E1)))
print("H_form(Eref)    :", f5(Href))
print("component dist  :", p)
print("H(p)            :", f5(entropy(p)))
print("forbidden parts :", zeros_of(p))
print("|psi|^2 == a    :", sci(maxdiff(last_a(), p)))
```

付録B. 記号類の割当と設計行列

B.1 記号 → 類

類	記号	意味
BIND	Σ, ∫ , Π, ∂ , ∇ , lim	束縛作用素（変数を捕獲する）
FN	ζ, β, Γ, ψ, sin, cos, log, exp, sqrt, H, softmax	名前付き関数
OP	+, −, ×, /, ^, =	中置演算子・関係
VAR	s, p, q, x, t, n, α, β, γ, i, a, θ, k, σ, m	自由・束縛変数（既定類）
NUM	0, 1, 2, 1/2, π, e	数・数学定数
DEL	( ) [ ] , _	区切り・添字

未登録の記号は VAR に落ちる。この既定は α 変換不変性（命題0）を壊さない：未知の名前は一律に同じ類へ写るため。

B.2 設計行列 W（部品 × 記号類）

部品	BIND	FN	OP	VAR	NUM	DEL	根拠（設計者の宣言）
chat_stream	0	1	0	2	0	1	変数の多い式は対話的な同定を要する
pdf_ingest	1	2	0	1	0	2	名前付き関数と添字が多い＝出典参照が必要
plot_panel	2	2	1	1	2	0	和・積分と定数＝数値プロットの対象
ledger_view	2	0	1	0	1	1	束縛作用素＝逐次的な commit の履歴
gate_editor	1	1	2	1	1	0	演算子密度＝ゲート列の編集
tomography	2	1	0	1	2	0	和と定数＝反復測定による推定
export_md	0	1	1	1	0	2	区切りの多い式＝整形出力の需要
settings	0	0	1	0	1	1	既定（残余）

この表はデータから推定されたものではない。§8-3 の限界がここに対応する。

付録C. 再現コード（Python 3、標準ライブラリのみ）

```
import math, cmath
from collections import Counter

CLASS = {}
def _c(cls, syms):
    for s in syms.split(): CLASS[s] = cls
_c('BIND','Sum Int Prod lim d/ds partial nabla')
_c('FN','zeta beta Gamma psi_dg sin cos log exp sqrt softmax H')
```

```
_c('OP','+ - * / ^ =')
_c('VAR','s p q x t n alpha beta_v gamma_v i a theta k sigma m')
_c('NUM','1 2 pi e 0 1/2')
_c('DEL','( ) [ ] , _')
klass = lambda t: CLASS.get(t, 'VAR')

def tokens(t):
    if isinstance(t, str): return [t]
    out = [t[0]]
    for c in t[1:]: out += tokens(c)
    return out

def depth_profile(t, d=0, acc=None):
    if acc is None: acc = Counter()
    acc[d] += 1
    if not isinstance(t, str):
        for c in t[1:]: depth_profile(c, d+1, acc)
    return acc

def arity_profile(t, acc=None):
    if acc is None: acc = Counter()
    if isinstance(t, str): acc[0] += 1
    else:
        acc[len(t)-1] += 1
        for c in t[1:]: arity_profile(c, acc)
    return acc

def shannon(counter):                       # x log x 核
    N = sum(counter.values())
    if N == 0: return 0.0
    return -sum((v/N) * math.log(v/N) for v in counter.values() if v > 0)

def form_entropy(t):                        # 式(5)
    tk = tokens(t)
    H0 = shannon(Counter(tk))
    H1 = shannon(Counter(klass(x) for x in tk))
    H2 = shannon(depth_profile(t))
    H3 = shannon(arity_profile(t))
    return (H0 + H1 + 0.5*(H2+H3)) / 3.0

def closed_form(a, dH, beta, phi):          # 式(7)
    w = [a[i]*(1.0 + math.cos(beta*dH*phi[i])) for i in range(len(a))]
    Z = sum(w)
    return [x/Z for x in w]

def two_branch(a, H1, H2, beta, phi):       # 式(6) 直接
    c = 1/math.sqrt(2)
    psi = [c*math.sqrt(a[i])*cmath.exp(1j*beta*H1*phi[i])
         + c*math.sqrt(a[i])*cmath.exp(1j*beta*H2*phi[i]) for i in range(len(a))]
    Z = sum(abs(x)**2 for x in psi)
    return [abs(x)**2/Z for x in psi]

CLS  = ['BIND','FN','OP','VAR','NUM','DEL']
COMP = ["chat_stream","pdf_ingest","plot_panel","ledger_view",
        "gate_editor","tomography","export_md","settings"]
WMAT = {"chat_stream":[0,1,0,2,0,1], "pdf_ingest":[1,2,0,1,0,2],
        "plot_panel":[2,2,1,1,2,0], "ledger_view":[2,0,1,0,1,1],
        "gate_editor":[1,1,2,1,1,0], "tomography":[2,1,0,1,2,0],
        "export_md":[0,1,1,1,0,2],   "settings":[0,0,1,0,1,1]}

def phi_of(t):                              # 式(8)
    tk = tokens(t); c = Counter(klass(x) for x in tk); N = len(tk)
    f = [c.get(k,0)/N for k in CLS]
    raw = [sum(WMAT[comp][j]*f[j] for j in range(6)) for comp in COMP]
    m = sum(raw)/len(raw)
    s = max(abs(x-m) for x in raw) or 1.0
    return [(x-m)/s for x in raw]
```

```
# 方程式は S 式で与える（本文 表1 の E1）
E1 = ('=', ('zeta','s'),
      ('*', ('^','2','s'),
       ('*', ('^','pi',('-','s','1')),
        ('*', ('sin',('/',('*','pi','s'),'2')),
         ('*', ('/',('beta','s',('-','1','s')),('Gamma','s')),
               ('zeta',('-','1','s')))))))
E4 = ('=','psi_dg',('*',('sqrt','a'),('exp',('*','i','theta'))))

prior = [1/8]*8
dH    = form_entropy(E1) - form_entropy(E4)
# 分解ゲイン。丸めた 0.24277 ではなく計算値そのものを使うこと。
# 引数がちょうど pi になって初めて cos(pi) = -1.0 が倍精度で厳密になり、
# 厳密零（禁止部品）が生成される（本文 4.3 の注記）。
beta  = math.pi / abs(dH)
p     = closed_form(prior, dH, beta, phi_of(E1))
print("H_form(E1) =", round(form_entropy(E1), 5))
print("dH_max     =", round(abs(dH), 5))
print("beta       =", round(beta, 5))
print("p =", [round(x, 4) for x in p])
print("forbidden  =", [i for i, x in enumerate(p) if x == 0.0])
```

付記（誠実性）

本稿が肯定するのは次の限定的・検証可能な事実のみである：(i) 有限の記号列と有限順序木に対して Shannon エントロピーは定義され、名前の付け替えに不変である；(ii) エンジンの位相核において単一分岐の位相ステップは出力分布を変えない（倍精度で 1.11×10^{-16}）；(iii) 共通位相モジュール下の等重み二分岐干渉は式(7)の閉形式を厳密に持つ（倍精度で 2.22×10^{-16}）；(iv) その因子は非負であり、厳密零を復活させず、引数が π のとき新たな厳密零を作る。

本稿が肯定**しない**ことも同様に明示する：`H_form` が方程式の数学的内容・深さ・重要性を測るという主張、生成されたアプリケーションが「良い」という主張、設計行列 W が客観的であるという主張、そして `ζ(s) = β(p,q)/log x`。最後のものは既存の監査で数値的に偽と確認されており、本稿はこれに一切依拠しない。第II部の予想1〜3 は厳密な定義を欠いており、現時点でアナロジーである。

参考文献

1. M. Yamaguchi, *Reviser-Extensible Grammars: A Q#-Targeted Quantum Front End for the Unknown-Prior Engine, Compiled and Interpreted in Bada*, 私家版.
2. M. Yamaguchi, *Bada — The Unknown-Prior Engine Language*（完全ソース：lexer, parser, interpreter, Bada→C compiler）, 私家版.
3. M. Yamaguchi, *QuantumBada — a generative-AI OS for a gate-model quantum computer*（ソース）, 私家版.
4. M. Yamaguchi, 「ベータ関数の商代数的対数構造とゼータ関数差分によるオイラー定数の出現」, 2026年7月21日.
5. M. Yamaguchi, 「ベータ関数とゼータ関数の構造的対応 — 関数等式・自明な零点・臨界線 Re s = 1/2 に関する証明可能な部分と予想の分離」, 2026年7月22日（改訂版）.
6. M. Yamaguchi, 「ナッシュ・シャノン公式に基づくゼータ関数と本物のゼータの相関」, 2026年6月.
7. M. Yamaguchi, *Euler product estrade from Heisenberg Non-commutative with deprivate equation*, 私家版.
8. C. E. Shannon, "A Mathematical Theory of Communication", *Bell System Technical Journal*, 27, 1948.
9. E. T. Jaynes, "Information Theory and Statistical Mechanics", *Physical Review*, 106, 1957.
10. J. Nash, "Equilibrium Points in n-Person Games", *PNAS*, 36, 1950.
11. K. Svore et al., "Q#: Enabling Scalable Quantum Computing and Development with a High-level Domain-specific Language", *RWDSL*, 2018.

全数値は当該環境の Python 3 標準ライブラリ（IEEE 倍精度）による。付録Cのコードで再現可能。文献は原典で確認されたい。